AEC1 · Cómo está resuelta

Paradigmas de la Programación · Angel Panadero Rodriguez · Common LISP (SBCL)

Guía para replicar a mano el fichero `angel_panadero_rodriguez_AEC1.lsp`. Aquí va la idea de cada ejercicio y su código sin comentarios; las respuestas teóricas completas están en los comentarios del propio fichero.

1. Cómo ejecutarlo

SBCL ya está instalado y en el PATH (abre una terminal nueva para que lo detecte). Desde la carpeta AEC1:

```
sbcl --script angel_panadero_rodriguez_AEC1.lsp
```

Muestra el resultado de cada ejemplo y, en el ejercicio 9, te pide un número. Para probar funciones sueltas, abre `sbcl` y haz `(load "angel_panadero_rodriguez_AEC1.lsp")`. Sales con `(exit)`.

2. El LISP que necesitas

`(defun f (a b) cuerpo)`	Define una función. Devuelve el valor de la última expresión.
`(car l) / (cdr l)`	Cabeza / resto de la lista: (car '(1 2 3)) → 1, (cdr '(1 2 3)) → (2 3).
`(cons x l)`	Lista nueva con x delante: (cons 1 '(2 3)) → (1 2 3).
`(list a b)`	Crea una lista. La uso como «tupla»: (list 1 2) → (1 2).
`(null l)`	¿Está vacía? Es la condición de los casos base.
`(if c si no)`	Condicional de dos ramas.
`(cond (c1 r1) (c2 r2) (t r))`	Varias ramas en orden; t es el «else».
`(let ((x valor)) cuerpo)`	Nombre local para un valor (no se reasigna).
`&optional (pos 1)`	Parámetro opcional con valor inicial (para acumuladores).
`#'f y (funcall f x)`	Pasar una función como dato / llamar a una función recibida.
`(lambda (x) (* 2 x))`	Función anónima.
`(mapcar f l) / (reduce f l)`	map y reduce de LISP.
`(format t "texto~%")`	Imprime en pantalla; ~% es salto de línea.
`'(1 2 3) #\a`	Lista literal (quote, no se evalúa) / carácter a.

3. La plantilla que se repite

Casi todos los ejercicios son recursión sobre listas: resuelves la cabeza (`car`) y llamas a la misma función con el resto (`cdr`) hasta llegar al caso base. Antes de escribir, responde a dos preguntas: **¿qué devuelvo con la lista vacía (o con un solo elemento)?** y **¿cómo combino la cabeza con el resultado del resto?**

```
(defun f (lista)
  (if (null lista)
      valor-caso-base                          ; caso base
      (combinar (car lista) (f (cdr lista)))))   ; caso recursivo
```

4. Ejercicio a ejercicio

Ejercicio 1 · imprimir_bienvenida

Un `format t` con el texto y `~%` al final. Es la única función impura (escribe en pantalla); todas las demás son puras.

```
(defun imprimir_bienvenida ()
  (format t "¡Bienvenidos al Paradigma Funcional: Inmutabilidad y Funciones Puras!~%"))
```

Ejercicio 2 · longitud_lista

Caso base: la lista vacía mide 0. Caso recursivo: 1 + la longitud del resto. No mira los elementos, así que vale para listas de cualquier tipo.

```
(defun longitud_lista (lista)
  (if (null lista)
      0
      (+ 1 (longitud_lista (cdr lista)))))

; (longitud_lista '(4 8 15)) = 1 + (1 + (1 + 0)) = 3
```

Ejercicio 3 · encontrar_minimo

Caso base: si queda un solo elemento (`(null (cdr lista))`), ese es el mínimo. Caso recursivo: calculo el mínimo del resto y me quedo con el menor entre él y la cabeza. El `let` guarda ese mínimo para no hacer la llamada recursiva dos veces.

```
(defun encontrar_minimo (lista)
  (if (null (cdr lista))
      (car lista)
      (let ((minimo_resto (encontrar_minimo (cdr lista))))
        (if (< (car lista) minimo_resto)
            (car lista)
            minimo_resto))))
```

Ejercicio 4 · encontrar_minimo_y_posicion

Mismo esquema que el 3, pero cada llamada devuelve la «tupla» `(valor posición)`. La posición va en un parámetro `&optional (pos 1)` que aumenta en 1 en cada llamada (el patrón de `primer-par-pos` de la Guía LISP). Comparo la cabeza con el valor del resultado del resto (`(car resultado_resto)`). Uso `<=` para que, si el mínimo se repite, gane la primera aparición.

```
(defun encontrar_minimo_y_posicion (lista &optional (pos 1))
  (cond
    ((null lista) nil)
    ((null (cdr lista)) (list (car lista) pos))
    (t (let ((resultado_resto (encontrar_minimo_y_posicion (cdr lista) (+ pos 1))))
         (if (<= (car lista) (car resultado_resto))
             (list (car lista) pos)
             resultado_resto)))))
```

Traza con '(9 1 7 3), de dentro hacia fuera: (3) en pos 4 → (3 4) · 7 contra 3 → (3 4) · 1 contra 3 → (1 2) · 9 contra 1 → **(1 2)**.

Ejercicio 5 · sumar_max_y_min (refactorización)

Mínimo y máximo solo se diferencian en el operador (`<` o `>`). Copio `encontrar_minimo`, la renombro a `encontrar_extremo`, le añado el parámetro `comparador` y cambio `(< a b)` por `(funcall comparador a b)`. Después la llamo con `#'>` y con `#'<` y sumo.

```
(defun encontrar_extremo (lista comparador)
  (if (null (cdr lista))
      (car lista)
      (let ((extremo_resto (encontrar_extremo (cdr lista) comparador)))
        (if (funcall comparador (car lista) extremo_resto)
            (car lista)
            extremo_resto))))

(defun sumar_max_y_min (lista)
  (+ (encontrar_extremo lista #'>)
     (encontrar_extremo lista #'<)))
```

Ejercicio 6 · obtener_caracter_n

Avanzo por la lista restando 1 a n. Cuando n vale 1, la cabeza es el elemento buscado. Si la lista se acaba antes, la posición no existe y devuelvo `nil`. Los caracteres se escriben `#\a`.

```
(defun obtener_caracter_n (lista n)
  (cond
    ((null lista) nil)
    ((= n 1) (car lista))
    (t (obtener_caracter_n (cdr lista) (- n 1)))))

; (obtener_caracter_n '(#\a #\b #\c #\d) 2)  => #\b
```

Ejercicio 7 · obtener_trinomio_n

Es el mismo algoritmo que el 6. Lo escribo una sola vez como `obtener_elemento_n` y `obtener_trinomio_n` se limita a llamarla. Cada trinomio es una lista `(1 2 3)`.

¿Se puede generalizar? Sí: la función nunca mira el tipo del elemento y LISP tiene tipado dinámico, así que vale para cualquier tipo (polimorfismo). *Ojo: el ejemplo del enunciado pone `obtener_caracter_n` por error; lo indico en un comentario.*

```
(defun obtener_elemento_n (lista n)
  (cond
    ((null lista) nil)
    ((= n 1) (car lista))
    (t (obtener_elemento_n (cdr lista) (- n 1)))))

(defun obtener_trinomio_n (lista n)
  (obtener_elemento_n lista n))
```

Ejercicio 8 · duplicar_lista

a) Recursiva: `cons` del doble de la cabeza con el resto ya duplicado. **b)** `mapcar` con una lambda. Como dos funciones no pueden llamarse igual, la b) se llama `duplicar_lista_map`. En ambos casos se crea una lista nueva y la original no cambia.

```
(defun duplicar_lista (lista)
  (if (null lista)
      nil
      (cons (* 2 (car lista)) (duplicar_lista (cdr lista)))))

(defun duplicar_lista_map (lista)
  (mapcar (lambda (x) (* 2 x)) lista))
```

c) Una lambda es una función anónima que se define donde se usa. Ventajas: se pasa directamente a funciones de orden superior (mapcar, reduce…), evita funciones auxiliares de un solo uso, puede capturar variables de su entorno (clausura) y hace el código más declarativo.

Ejercicio 9 · repetir_accion_n

Si n > 0, ejecuto la acción con `funcall` y me llamo con n-1; si no, paro. Uso `when` en vez de `if` porque permite poner dos expresiones seguidas en el cuerpo. La demo lee el número con `(read)` y pasa la función con `#'imprimir_bienvenida`. `finish-output` hace que la pregunta aparezca antes de esperar la respuesta.

```
(defun repetir_accion_n (accion n)
  (when (> n 0)
    (funcall accion)
    (repetir_accion_n accion (- n 1))))

(defun demo_repetir_bienvenida ()
  (format t "~%¿Cuántas veces quieres mostrar la bienvenida? ")
  (finish-output)
  (repetir_accion_n #'imprimir_bienvenida (read)))

(demo_repetir_bienvenida)
```

Ejercicio 10 · map/reduce y el lenguaje

Incluyo un ejemplo pequeño: cada sublista representa un bloque de datos en un nodo; `mapcar` cuenta cada bloque (reutilizando `longitud_lista`) y `reduce` suma los parciales.

```
(defun contar_elementos_map_reduce (bloques)
  (reduce #'+ (mapcar #'longitud_lista bloques)))

; (contar_elementos_map_reduce '((1 2 3) (4 5) (6 7 8 9)))  => 9
```

Ideas clave de las respuestas (desarrolladas en el fichero):

- **a)** Un for es secuencial y modifica un acumulador compartido. map procesa cada elemento de forma independiente, así que se puede repartir entre nodos en paralelo; reduce combina con una operación asociativa, así que cada nodo reduce su parte y luego se juntan. Ventajas: escalabilidad, sin condiciones de carrera, tolerancia a fallos (una función pura se puede reejecutar en otro nodo) y código declarativo.
- **b)** Common LISP: multiparadigma con núcleo funcional, S-expresiones con notación prefija, código = datos (homoiconicidad), recolector de basura, REPL, y espacios de nombres separados para funciones y variables (por eso se usan `#'` y `funcall`). Tipado **dinámico** (se comprueba al ejecutar) y **fuerte** (`(+ 1 "a")` da error). Implica más flexibilidad y funciones genéricas gratis, a cambio de que los errores de tipo aparezcan al ejecutar y no al compilar.
- **Por qué los lenguajes modernos lo incorporan:** concurrencia y multinúcleo (la inmutabilidad la hace más segura), código más conciso con map/filter/reduce y lambdas, funciones puras fáciles de probar, y encaje natural con big data y streams.

5. Antes de entregar

- Nombre del fichero exacto: `angel_panadero_rodriguez_AEC1.lsp`.
- Cabecera con `; Paradigmas de la Programación`, `; AEC1` y `; Nombre: ...`, y un comentario `; Ejercicio N` antes de cada ejercicio.
- Nombres de funciones idénticos a los del enunciado, con guion bajo. SBCL los muestra en mayúsculas, pero LISP no distingue mayúsculas de minúsculas.
- Respuestas a las preguntas (7, 8c, 10a, 10b) como comentarios dentro del fichero.
- Ejecútalo una vez con `sbcl --script` y comprueba que no da errores.